ДОМАШНЕЕ ЗАДАНИЕ / ИНЖЕНЕРИЯ ДАННЫХ

Аналитика IT-вакансий в RustFS

Собственный MapReduce, независимый SQL-расчёт DuckDB, анализ зарплат и регионов, профилирование времени и памяти.

Автор	Индивидуальный сегмент
Нимеева Ангелина Борисовна	DevOps-инженер Фильтрация по столбцу role

Дата выполнения: 21 октября 2026 года. Время локальных запусков указано по Москве; суффиксы объектов S3 сформированы в UTC.

Основные результаты

Показатель	Полученный результат
Исходные данные	229 388 строк; 847 619 384 байта
Выборка своего варианта	5 158 вакансий; 1 624 без навыков
Полное совпадение методов	1 401 навык, 20 454 пары, 144 024 тройки
Зарплаты и регионы	762 сопоставимые зарплаты; 3 региона
Профилирование и отказ	По 3 попытки; управляемый отказ и полный перезапуск

Репозиторий: https://github.com/shikar1/vk-data-engineering-homework

Отчёт построен по сохранённым результатам фактических запусков. Скриншоты предоставлены автором; таблицы и графики рассчитаны по исходным CSV/JSON артефактам.

1. Хранилище и исходные данные

Использован ранее развёрнутый контейнер rustfs-lab2. S3 доступен на localhost:9000, консоль - localhost:9001. Программы работают в Docker и обращаются к S3 через host.docker.internal:9000.

Объект	Значение
Бакет	hh-lake
Исходный файл	it_vacancies_2024.csv
Объём	847 619 384 байта (около 808,35 МиБ)
Роль	DevOps-инженер
Отбор	Точное равенство role после очистки пробелов и нормализации регистра

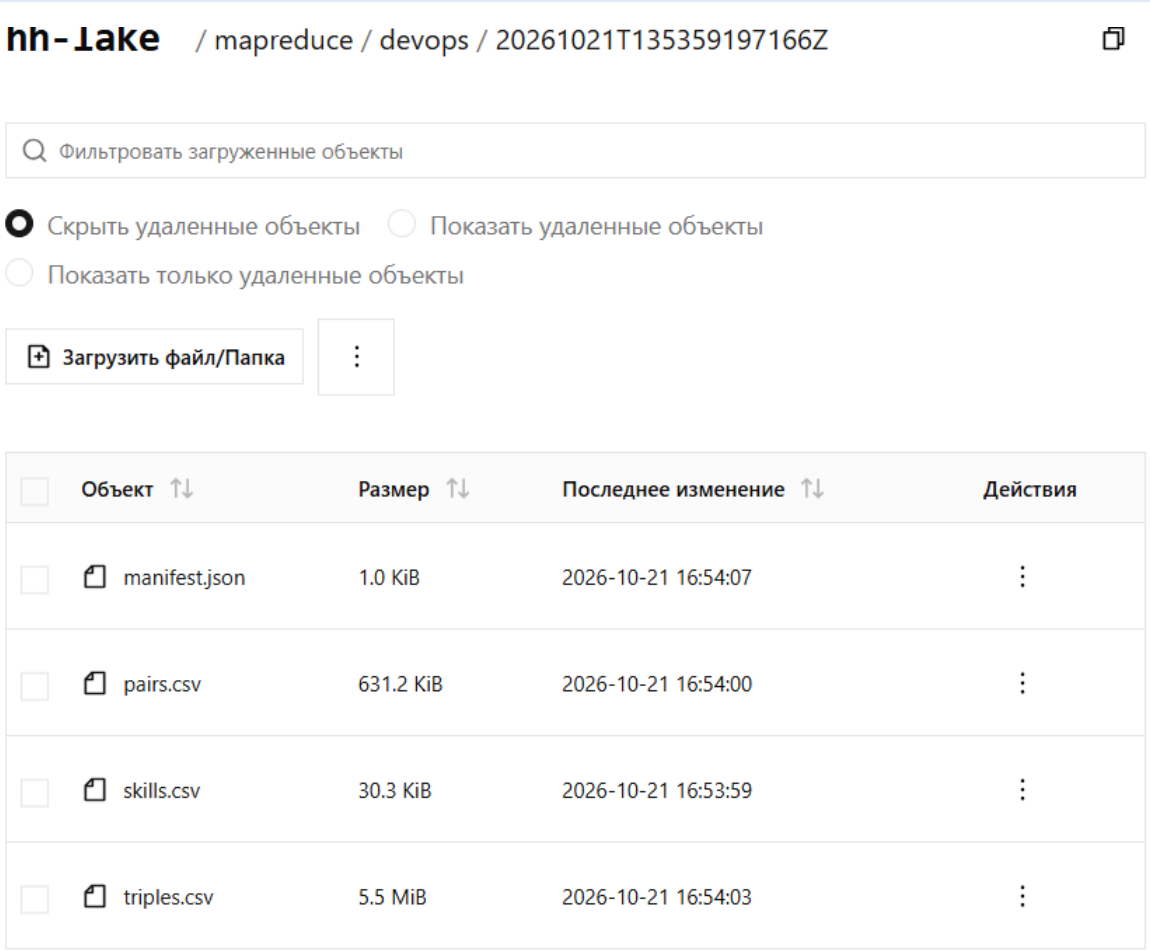


Рисунок 1. Фактический список выходных объектов MapReduce в консоли RustFS: skills.csv, pairs.csv, triples.csv и manifest.json.

Ключи доступа берутся из окружения работающего контейнера и передаются дочернему Docker-процессу через окружение. Их значения в коде, отчёте и логах не сохраняются. Python в Windows для расчётов не требуется.

2. Собственная реализация MapReduce

Исходный CSV читается непосредственно из HTTP-ответа S3 потоково. Полный исходный файл не сохраняется на локальный диск и не загружается целиком в Python-объект. В памяти остаются счётчики уникальных сочетаний. Реализация однопроцессная, без Hadoop/Spark и распределённого планировщика.

- Map: для каждой подходящей строки разделить key_skills по ';', схлопнуть пробелы, применить casefold, убрать дубликаты внутри вакансии и упорядочить навыки.

- Сформировать неупорядоченные сочетания размеров 1, 2 и 3 с единичным вкладом. Reduce суммирует вклады в Counter.

- Проверить общее число строк и выбранных вакансий, суммы вкладов C(n,k), затем отсортировать таблицы по убыванию частоты и названию навыков.

- Сохранить CSV в S3, скачать их повторно, сравнить байты и разобранные частоты; записать манифест с SHA256.

Размер стека	Уникальных сочетаний	Сумма частот
1	1 401	28 336
2	20 454	134 651
3	144 024	506 549

Вакансия с n различными навыками создаёт C(n,1)=n, C(n,2)=n(n-1)/2 и C(n,3)=n(n-1)(n-2)/6 вкладов. Например, для 10 навыков это 10, 45 и 120. Поэтому пар и троек существенно больше, хотя вакансии те же.

Наиболее частые навыки

Навык	Вакансий	Навык	Вакансий
linux	2 190	python	1 058
kubernetes	1 862	ci/cd	1 044
docker	1 850	bash	925
devops	1 319	postgresql	892
ansible	1 252	grafana	830

Частоты считаются по исходным строкам. Повторяющиеся vacancy_id не удаляются: это сохраняет единую единицу наблюдения и контроль 5 158 строк своего варианта.

3. Независимый расчёт DuckDB и сверка

DuckDB 1.4.3 с расширением httpfs самостоятельно читает тот же CSV по S3 URI. В SQL явно заданы формат CSV и 23 строковых столбца: это исключает ошибки автоопределения кавычек и типов. Каждой исходной строке назначается row_id.

```
FROM read_csv('s3://hh-lake/it_vacancies_2024.csv', ...)

CREATE TEMP TABLE selected AS
SELECT row_id, key_skills FROM raw
WHERE lower(trim(role)) = 'devops-инженер';

SELECT first_skill.skill AS skill_1,
       second_skill.skill AS skill_2, count(*) AS count
FROM vacancy_skills AS first_skill
JOIN vacancy_skills AS second_skill
  ON first_skill.row_id = second_skill.row_id
 AND first_skill.skill < second_skill.skill
GROUP BY first_skill.skill, second_skill.skill;
```

После разбиения key_skills через string_split/UNNEST удаляются дубликаты навыков в пределах row_id. Пары и тройки формируются соединениями с условиями skill_1 < skill_2 < skill_3; подсчёт выполняет GROUP BY. Полный исполняемый запрос находится в duckdb.sql.

Таблица	Строк	Сумма частот	Различий
skills	1 401	28 336	0
pairs	20 454	134 651	0
triples	144 024	506 549	0

```
FULL_COMPARISON: PASS
TOTAL: 229388
MATCHED: 5158
WITHOUT_SKILLS: 1624
```

Сверка выполняется через EXCEPT ALL в обе стороны, по всем столбцам и частотам. Эталон скачивается из S3 и проверяется по SHA256. Во всех трёх таблицах ноль различий; проверено 165 879 уникальных сочетаний, а не только топ-10.

MapReduce использует casefold, SQL - lower и нормализацию Unicode-пробелов. Для произвольных строк это не полностью одинаковые операции; совпадение подтверждено именно на данном наборе полным сравнением результатов.

4. Методика зарплатного анализа

Зарплаты сравниваются только внутри своего варианта DevOps. Основная подвыборка: RUR/RUB с gross=false. Зарплаты с gross=true и суммы в других валютах не смешиваются; налоговая и валютная конвертация не выполняется.

Причина / группа	Число строк
Зарплата отсутствует	4 047
Включены в сопоставимую выборку	762
Другой налоговый признак	275
Другая валюта	74

Если указаны обе границы, оценка зарплаты равна (salary_from + salary_to)/2. Если указана одна граница, используется она. Некорректные, неположительные, бесконечные и перевёрнутые вилки исключаются. Одна граница - приближение зарплаты, а не точное предложение.

Показатель	Значение
Сопоставимых зарплат	762
Из них вакансий с навыками	564
Общая медиана оценки зарплаты	210 000 руб.
Допущено в рейтинги	236 пар; 688 троек

Как выбирается база для дополнительного навыка

Для пары базой служит более частый из двух навыков. Для тройки - более частая из трёх входящих пар. Частота базы определяется по всем 5 158 строкам роли; при равенстве используется лексикографический порядок. Базовая выборка включает вакансии расширенного сочетания.

```
difference_rub = median(combination) - median(baseline)
difference_pct = 100 * (median(combination) / median(baseline) - 1)
```

Оба размера зарплатной подвыборки должны быть не меньше 20. В таблицах сохраняются n сочетания, n базы, обе медианы, добавленный навык и разница. Порог 20 ограничивает низкую поддержку, но не является тестом статистической значимости.

5. Сочетания с более высокой медианой

Первые пять результатов отдельно для пар и троек, упорядоченные по процентной разнице медиан относительно выбранной базы. n / n базы - число сопоставимых зарплат.

Пары

Сочетание / добавлен	База	Медиана, руб.	Медиана базы	Разница, %	n / n базы
devops + terraform Добавлен: terraform	devops	295 000	200 000	+47,50	46 / 254
devops + elk Добавлен: elk	devops	257 500	200 000	+28,75	54 / 254
kubernetes + linux Добавлен: kubernetes	linux	250 000	200 000	+25,00	200 / 365
ansible + linux Добавлен: ansible	linux	250 000	200 000	+25,00	141 / 365
linux + prometheus Добавлен: prometheus	linux	250 000	200 000	+25,00	104 / 365

Тройки

Сочетание / добавлен	База	Медиана, руб.	Медиана базы	Разница, %	n / n базы
bash + devops + elk Добавлен: elk	bash + devops	350 000	230 000	+52,17	22 / 88
devops + linux + terraform Добавлен: terraform	devops + linux	300 000	200 000	+50,00	33 / 182
ansible + git + gitlab Добавлен: gitlab	ansible + git	300 000	212 500	+41,18	23 / 68
ansible + devops + gitlab Добавлен: gitlab	ansible + devops	300 000	215 000	+39,53	37 / 99
devops + gitlab + linux Добавлен: gitlab	devops + linux	275 000	200 000	+37,50	61 / 182

Интерпретация: различается состав вакансий с данным сочетанием и базой. Наблюдаемая разница не доказывает, что добавление навыка само по себе изменяет зарплату; опыт, работодатель и регион здесь не контролируются.

6. Сочетания с более низкой медианой

Первые пять результатов отдельно для пар и троек, упорядоченные по процентной разнице медиан относительно выбранной базы. n / n базы - число сопоставимых зарплат.

Пары

Сочетание / добавлен	База	Медиана, руб.	Медиана базы	Разница, %	n / n базы
postgresql + zabbix Добавлен: zabbix	postgresql	192 500	250 000	-23,00	48 / 199
nginx + postgresql Добавлен: nginx	postgresql	200 000	250 000	-20,00	84 / 199
git + grafana Добавлен: git	grafana	200 000	250 000	-20,00	53 / 172
git + python Добавлен: git	python	180 000	225 000	-20,00	53 / 146
ansible + zabbix Добавлен: zabbix	ansible	200 000	250 000	-20,00	48 / 200

Тройки

Сочетание / добавлен	База	Медиана, руб.	Медиана базы	Разница, %	n / n базы
git + grafana + prometheus Добавлен: git	grafana + prometheus	160 000	250 000	-36,00	30 / 99
postgresql + prometheus + zabbix Добавлен: zabbix	postgresql + prometheus	160 000	250 000	-36,00	23 / 66
ansible + postgresql + zabbix Добавлен: zabbix	ansible + postgresql	167 500	250 000	-33,00	24 / 84
devops + git + grafana Добавлен: git	devops + grafana	145 000	210 000	-30,95	30 / 92
bash + docker + git Добавлен: git	bash + docker	155 000	222 500	-30,34	40 / 110

Интерпретация: различается состав вакансий с данным сочетанием и базой. Наблюдаемая разница не доказывает, что добавление навыка само по себе изменяет зарплату; опыт, работодатель и регион здесь не контролируются.

7. Региональный анализ: объёмы и зарплаты

Выбраны три наиболее представленных значения area: Москва, Санкт-Петербург и Екатеринбург. Это точные метки городов в данных, без объединения с областями. Вместе они содержат 3 881 из 5 158 строк DevOps (75,24%).

Регион	Вакансий	С навыками	Зарплат	Покрытие	Медиана, руб.
Москва	3 007	2 015	406	13,50%	250 000
Санкт-Петербург	686	488	103	15,01%	220 000
Екатеринбург	188	136	34	18,09%	150 000

Лидеры спроса каждого размера стека

Регион	Размер	Сочетание	Частота	Доля, %	n зарплат
Москва	1	linux	1 210	40,24	177
Москва	2	docker + kubernetes	700	23,28	109
Москва	3	docker + kubernetes + linux	436	14,50	70
Санкт-Петербург	1	linux	299	43,59	51
Санкт-Петербург	2	docker + kubernetes	175	25,51	35
Санкт-Петербург	3	docker + kubernetes + linux	110	16,03	25
Екатеринбург	1	linux	91	48,40	20
Екатеринбург	2	docker + kubernetes	64	34,04	18
Екатеринбург	3	docker + kubernetes + linux	49	26,06	12

Доля считается от всех DevOps-вакансий региона, включая записи без навыков и без зарплаты. Медианы - только по сопоставимой рублёвой подвыборке с gross=false; правила совпадают с общим анализом.

У ведущей тройки Екатеринбурга только 12 зарплатных наблюдений, у пары - 18. Такие медианы сохранены с размерами выборок, но имеют признак недостаточной поддержки (<20) и не считаются устойчивым зарплатным рейтингом.

8. Региональные различия и проверки

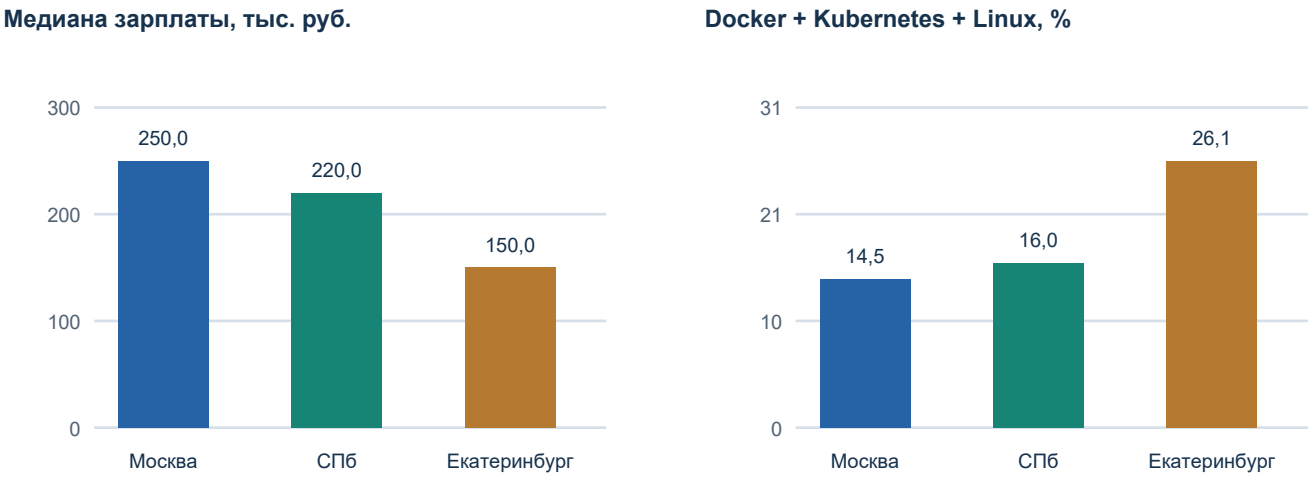


Рисунок 2. Слева - медианы сопоставимых зарплат; справа - доля одного и того же сочетания во всех трёх регионах. Источник: regional-overview.csv и regional-top-stacks.csv.

- Москва имеет медиану 250 000 руб., Санкт-Петербург - 220 000 руб., Екатеринбург - 150 000 руб. Разница Москвы и Екатеринбурга составляет 100 000 руб.; зарплатные подвыборки различаются по размеру: 406 против 34.

- Во всех трёх регионах лидируют linux, пара docker + kubernetes и тройка docker + kubernetes + linux. При этом относительная доля тройки максимальна в Екатеринбурге (26,06%), несмотря на меньший абсолютный объём вакансий.

- Среди глобального топ-10 отдельных навыков наибольший региональный разрыв долей у docker: 42,02% в Екатеринбурге и 33,29% в Москве, разница 8,73 процентного пункта.

- Навыки не указаны в 32,99% московских записей, 28,86% петербургских и 27,66% екатеринбургских. Отсутствие навыков в записи нельзя трактовать как отсутствие требований работодателя.

Проверка	Результат
Независимый Python-перебор всех региональных сочетаний	169 854 частоты; различий 0
Проверка медиан регионов и их топов	3 региональные медианы и 90 медиан топов совпали
Сопоставление одинаковых сочетаний	Глобальные топ-10 для каждого размера стека во всех регионах, включая нулевые частоты

Расчёт описательный. Структура опыта, работодатели и удалённая работа не контролируются; причинный эффект региона не установлен.

9. Профилирование: единый протокол

Выполнено по три попытки каждого метода в свежих дочерних Python-процессах. Порядок чередовался по раундам: MapReduce/DuckDB, DuckDB/MapReduce, MapReduce/DuckDB. Все шесть результатов совпали по SHA256 трёх полных канонических CSV.

Условие	Значение
Среда	Python 3.12.15; DuckDB 1.4.3; Linux/WSL2 x86_64
Ресурсы контейнера	2 CPU; 2 ГиБ RAM. DuckDB: threads=2, memory_limit=1GB; CSV parallel=false. MapReduce однопоточный.
Время	perf_counter родителя: создание дочернего процесса - его завершение
Память	Пик RSS за всю жизнь дочернего процесса через Linux wait4; единица - МиБ
Включено	Импорты, подключение, чтение S3, нормализация, все три агрегации, сортировка и локальная запись CSV
Не включено	Сборка/запуск Docker, сверка эталона, загрузка выходов в S3, тест отказа и перезапуск; память RustFS и контроллера

Исходные измерения

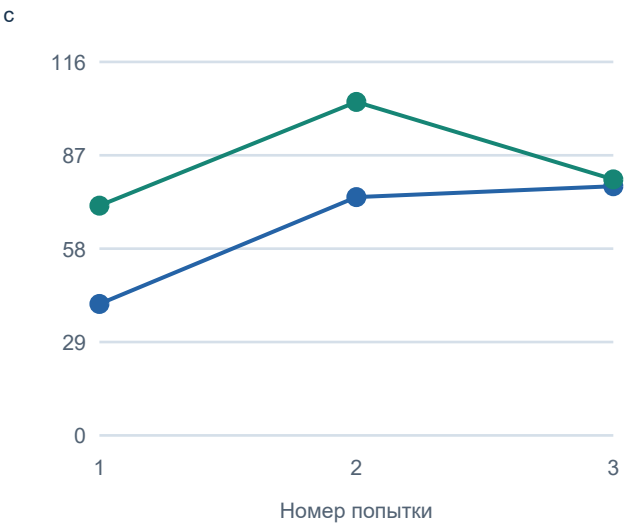
Метод	Попытка	Время, с	Пик RSS, МиБ	Совпадение
mapreduce	1	40,938	79,41	да
duckdb	1	71,552	1 010,16	да
duckdb	2	103,853	963,74	да
mapreduce	2	74,214	79,25	да
mapreduce	3	77,600	79,19	да
duckdb	3	79,729	966,52	да

Кеши ОС/RustFS не очищались; источник уже читался ранее. Это повторные замеры текущего окружения, без утверждения о холодном кеше. Перед, между и после запусков контролировались ETag, размер и Last-Modified источника; изменений не обнаружено. Multipart ETag не трактуется как SHA256 файла.

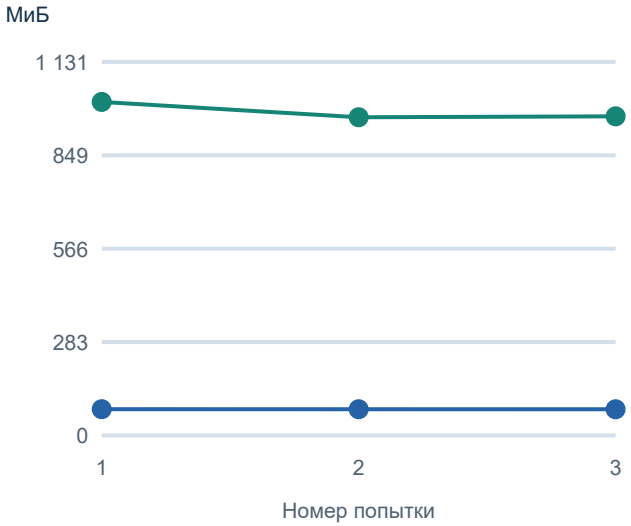
10. Время и память: результаты

Метод	Медиана времени, с	Мин - макс, с	SD времени, с	Медиана RSS, МиБ	Макс. RSS, МиБ
mapreduce	74,214	40,94 - 77,60	20,26	79,25	79,41
duckdb	79,729	71,55 - 103,85	16,79	966,52	1 010,16

Время полного процесса



Пик RSS процесса



Синий - MapReduce; зелёный - DuckDB. Каждая точка соответствует одной фактической попытке; номер попытки не означает последовательный запуск только одного метода.

```
method    attempt wall_seconds       peak_rss_mib  all_tables_equal
------    ------- ------------       ------------  ----------------
mapreduce 1       40.938290570999925 79.41015625   True
duckdb    1       71.55215661299962  1010.16015625 True
duckdb    2       103.85314838100021 963.73828125  True
mapreduce 2       74.21398368600012  79.25         True
mapreduce 3       77.60047604900001  79.19140625   True
duckdb    3       79.7294551519999   966.515625    True
```

Рисунок 3. Предоставленный автором скриншот вывода PowerShell с шестью измерениями и проверкой all_tables_equal=True.

Медиана времени: 74,214 с у MapReduce и 79,729 с у DuckDB. Диапазоны перекрываются, разброс велик; устойчивое превосходство по скорости тремя попытками не доказано. Медиана пиков RSS у MapReduce примерно в 12,20 раза меньше (79,25 против 966,52 МиБ).

Разницу памяти могут объяснять CSV-буферы и SQL-операторы DuckDB по сравнению с потоковым разбором и счётчиками MapReduce. Отдельной трассировки аллокаций не было. Старый диагностический SQL-замер 39,436 с не смешивается с этой серией: его границы были другими.

11. Отказ маппера и восстановление

В реальную функцию mapper введён управляемый отказ на 100-й подходящей вакансии. Проверяется ошибка вычислительного этапа до публикации таблиц; отказ S3 во время загрузки здесь не моделируется.

```
python /app/mapreduce.py --fail-after-matches 100

FAILED: RuntimeError: INJECTED_MAPPER_FAILURE
```

Проверка	Факт
Код завершения при отказе	1
Локальные CSV/manifest	Отсутствуют
S3 skills/pairs/triples/manifest в префиксе отказа	HEAD каждого объекта вернул 404
Повторный запуск без управляемой ошибки	Код 0; все три таблицы совпали с эталоном
Проверка восстановленных выходов	Загружены и перечитаны из S3

Запуск теста	Время, с	Пик RSS, МиБ
Отказ	1,629	33,52
Полный повторный запуск	81,642	150,03

Эти два запуска не входят в шесть сравнительных попыток. Перезапуск исполняет обычную программу с загрузкой и обратным чтением S3, поэтому его границы шире, чем у сравнительного worker.

```
TOTAL: 229388
MATCHED: 5158
WITHOUT_SKILLS: 1624
VERIFIED: all output objects downloaded and checked
MAPPER_FAILURE_AND_RECOVERY: PASS
```

Показаны корректное прерывание и явно инициированный полный перезапуск с начала. Автоматические повторы, контрольные точки и замещение узлов распределённого кластера не реализованы. Фактические логи: fault-failed.log и fault-recovered.log; измерения - fault-test.json.

12. Воспроизведение и комплект материалов

Предварительные условия: Docker Desktop, работающий rustfs-lab2, бакет hh-lake и исходный объект it_vacancies_2024.csv. В репозитории исходный CSV не дублируется. Ключи должны быть доступны в окружении контейнера RustFS.

```
Set-Location .\devops-lab

# Проверка столбцов и примеров исходного CSV
& .\run.ps1 -Inspect

# MapReduce и SQL с полной сверкой
& .\run.ps1
& .\run-duckdb.ps1

# Зарплаты и три региона
& .\run-analysis.ps1
& .\run-regions.ps1

# По 3 замера и тест отказа/перезапуска
& .\run-benchmark.ps1
```

Файлы	Назначение
mapreduce.py; run.ps1	Потоковый расчёт и работа с S3
duckdb.sql; duckdb_run.py	Независимый SQL и полная сверка
analysis-source.sql; salary.sql; analysis_run.py	Фильтрация зарплат, медианы и выбор базы
regions.sql; regional_analysis.py	Частоты, доли, зарплаты и независимая проверка регионов
benchmark_worker.py; benchmark_run.py	Свежие процессы, замеры и тест отказа
Dockerfile.duckdb; скрипты *.ps1	Воспроизводимое окружение без Python в Windows
build_report.py; evidence/; report/	Генератор PDF, оригинальные скриншоты и готовый отчёт

Репозиторий назначения: https://github.com/shikar1/vk-data-engineering-homework. Материалы работы размещаются в отдельной папке devops-lab. В комплект входят исходные программы, фактические таблицы и логи, настройки замеров и PDF.

13. Выводы и ограничения

- Построен рабочий пайплайн от S3-объекта RustFS до частот, зарплатных и региональных таблиц. Выбор варианта задан собственным условием по role и подтверждён числом 5 158.

- MapReduce и DuckDB полностью совпали по навыкам, парам и тройкам. Рост числа сочетаний объясняется C(n,k), а не увеличением количества вакансий.

- Самая высокая положительная разница среди поддержанных пар - devops + terraform относительно devops: +47,50%. Среди троек - bash + devops + elk относительно bash + devops: +52,17%.

- Есть и отрицательные разницы: postgresql + zabbix относительно postgresql -23%; git + grafana + prometheus относительно grafana + prometheus -36%. В итоговых таблицах явно названы добавленные навыки и медианы баз.

- Московская медиана сопоставимой зарплаты выше, но доли некоторых сочетаний выше в Екатеринбурге. Абсолютные частоты и относительная востребованность отвечают на разные вопросы.

- Три попытки каждого метода выполнены с согласованными границами. MapReduce показал существенно меньший пик RSS; близкие медианы времени при большом разбросе не доказывают устойчивого преимущества по скорости.

- Отказ маппера не создал незавершённых выходов; отдельный полный перезапуск восстановил правильный результат. Это не распределённая отказоустойчивость.

Ограничения

Зарплата отсутствует у 4 047 из 5 158 вакансий. Анализ использует 762 сопоставимые рублёвые зарплаты с gross=false, из них 564 с навыками. Вилки с одной границей приблизительны. Опыт, работодатель, региональные различия состава вакансий и удалённая работа не контролируются. Наблюдаемые разницы не являются причинными эффектами.

Бонусная оптимизация с отдельным сравнением 'до/после' не заявляется: сравнение двух подходов не подменяет эксперимент по оптимизации одного алгоритма.

Источники и доказательства

Данные: s3://hh-lake/it_vacancies_2024.csv; точный размер и ETag зафиксированы в configuration.json.

Результаты S3: mapreduce/devops/20261021T135359197166Z; duckdb/devops/20261021T140217322374Z; salary/devops/20261021T140856183615Z; regions/devops/20261021T141449855514Z; benchmark/devops/20261021T142021716249Z.

Документация DuckDB: https://duckdb.org/docs/stable/data/csv/overview и https://duckdb.org/docs/stable/core_extensions/httpfs/s3api. Расчёт использует закреплённую версию 1.4.3.

Скриншоты: реальные изображения консоли RustFS и вывода PowerShell, предоставленные автором. Остальные таблицы и графики автоматически построены из сохранённых результатов.